

Phebe: A Multi-Agent Natural Language Interface for Interactive Data Visualization

Guntawit Anurakboonying
Mahidol University International College
Phutthamonthon, Nakhon Pathom, Thailand
guntawit.anu@student.mahidol.edu

Chalantorn Sawangwongchinsri
Mahidol University International College
Phutthamonthon, Nakhon Pathom, Thailand
chalantorn.saa@student.mahidol.edu

Ratchaphon Proongpattanaskul
Mahidol University International College
Phutthamonthon, Nakhon Pathom, Thailand
ratchaphon.prn@student.mahidol.edu

Tanasanee Phienthrakul
Mahidol University
Phutthamonthon, Nakhon Pathom, Thailand
tanasanee.phil@mahidol.ac.th

Abstract

Most data visualization tools require technical expertise, creating significant accessibility barriers for non-technical users. We present PHEBE, a web-based platform that enables users to upload tabular datasets, perform AI-guided data cleaning, and generate interactive visualizations by expressing analytical needs in plain English. The system is built around a multi-agent architecture powered by large language models (LLMs), comprising an Orchestrator Agent for intent classification, a Text-to-SQL Agent with execution-based self-correction, an Analysis Agent for statistical computations, and a Chart Recommendation Agent for automated visual encoding selection. An end-to-end pipeline integrates CSV ingestion with robust validation, a six-category data quality subsystem, SQL generation against user-uploaded datasets stored in MySQL, and a D3.js-based rendering layer supporting seven chart types, a drag-and-drop dashboard, and PDF export. Evaluated on the BIRD-Mini Text-to-SQL benchmark (500 queries), PHEBE achieves 61.6% execution accuracy — an 8 percentage point improvement over the GPT baseline — with a 0% SQL syntax error rate attributable to the self-correction loop. A user survey further confirms that 85% of participants ($n = 27$) rated their ability to explore complex datasets without writing SQL or code at 4 or 5 out of 5, validating the platform’s goal of democratizing data exploration.

CCS Concepts

• **Information systems** → **Data analytics**; • **Human-centered computing** → **Visual analytics**; • **Computing methodologies** → *Natural language processing*.

Keywords

Natural Language Interfaces, Data Visualization, Text-to-SQL, Large Language Models, Multi-Agent Systems, Data Cleansing

ACM Reference Format:

Guntawit Anurakboonying, Ratchaphon Proongpattanaskul, Chalantorn Sawangwongchinsri, and Tanasanee Phienthrakul. 2026. Phebe: A Multi-Agent Natural Language Interface for Interactive Data Visualization. In *14th International Conference on Advances in Information Technology (IAIT 2026)*, June 17–19, 2026, Bangkok, Thailand. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3816713.3819784>

1 Introduction

Data visualization is foundational to modern data analysis, transforming complex datasets into interpretable representations that reveal patterns, trends, and outliers. Yet the process of creating effective visualizations demands proficiency in programming languages such as Python or JavaScript, or expertise in commercial business intelligence (BI) tools like Tableau and Power BI. These technical requirements create a significant accessibility barrier, preventing non-technical stakeholders — business analysts, managers, and domain experts — from independently exploring their own data.

Natural Language to Visualization (NL2Vis) has emerged as a research direction seeking to close this gap, letting users describe desired visualizations in plain English rather than through code or specialized query languages [11]. Unlike Text-to-SQL, which translates natural language to database queries only, NL2Vis must additionally determine how data should be visually represented: chart type, axis mappings, aggregation functions, and filtering conditions. Early systems such as NL4DV and FlowSense relied on rule-based semantic parsers with limited ability to handle diverse or ambiguous expressions. The advent of large language models (LLMs) such as GPT-4 has significantly advanced NL2Vis capabilities, demonstrating strong performance in understanding context and generating structured outputs mapped to visualization specifications [9].

Despite these advances, most NL2Vis research focuses on isolated pipeline components rather than delivering a fully functional, end-to-end application accessible to non-technical users. A further gap is that many systems assume data is already clean and well-structured, ignoring the reality that real-world datasets frequently require preprocessing before meaningful visualizations can be generated [4].

This paper presents PHEBE, a web-based platform that addresses both gaps. Users can upload their own CSV datasets, perform data



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

IAIT 2026, Bangkok, Thailand

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2436-7/26/06

<https://doi.org/10.1145/3816713.3819784>

preprocessing through an AI-guided interface, and generate interactive visualizations using natural language queries. PHEBE makes three primary contributions:

- (1) A **multi-agent NL2Vis architecture** composed of four co-operating LLM-powered agents (Orchestrator, Text-to-SQL, Analysis, and Chart Recommendation), with execution-based self-correction and intent-aware routing.
- (2) A **six-category AI-assisted data cleaning pipeline** that detects missing values, outliers, duplicate rows and columns, format inconsistencies, and high-cardinality columns, and presents AI-recommended remediation options to users via an interactive interface.
- (3) An **end-to-end empirical evaluation** on the BIRD-Mini benchmark (500 queries) and a user survey measuring accessibility for non-technical users across usability, clarity, usefulness, and overall satisfaction dimensions.

2 Related Work

2.1 Natural Language Interfaces for Data

The problem of translating natural language to structured queries has a long history. Semantic parsing approaches such as ATIS used handcrafted grammars that proved brittle across domains. The Text-to-SQL renaissance brought neural approaches — sequence-to-sequence models, graph neural networks, and eventually pre-trained transformers — that dramatically improved cross-domain generalization [12]. Comprehensive surveys [11] identify semantic parsing as the central technology enabling both Text-to-SQL and NL2Vis, noting that LLMs such as ChatGPT represent a paradigm shift: inference-only LLMs can reason about data schemas without task-specific fine-tuning.

2.2 NL2Vis Systems

Early rule-based NL2Vis systems such as NL4DV [7] provided a toolkit for generating Vega-Lite specifications from natural language, but relied on handcrafted intent parsers that break on out-of-vocabulary phrasings. Draco [6] formalized visualization design as a constraint satisfaction problem, enabling automated recommendation but requiring expert-authored rule weights rather than natural language input. FlowSense [10] extended NL interaction to dataflow graphs but remained limited to a fixed schema of supported intents.

Mirror [9] represents a more recent shift toward LLM-backed NL interfaces: it translates user queries into SQL, summarizes results, and renders visualizations, with a human-in-the-loop design allowing users to review and edit generated SQL. PHEBE extends this direction by additionally incorporating automated data cleaning and AI-driven chart recommendation, removing the need for users to inspect or correct raw SQL. The Text-to-Pipeline work [4] addresses NL-driven data preparation, constructing the PARROT benchmark of 18,000 pipelines; PHEBE’s data cleaning subsystem is complementary, focusing on interactive, user-confirmed remediation rather than fully automated pipeline generation.

2.3 Text-to-SQL Benchmarks and Prompting

The BIRD benchmark [5] provides 500 real-world question–SQL pairs across multiple database domains and difficulty levels, evaluated using execution accuracy (EX). Prompting strategies have evolved significantly: DIN-SQL [8] decomposes complex queries into sub-problems using few-shot chain-of-thought, while DAIL-SQL [3] selects question-specific demonstrations to improve cross-domain generalization. State-of-the-art systems such as Google’s Distillery ($\approx 75\text{--}77\%$), Apple’s CHASE-SQL ($\approx 73\%$), and SuperSQL ($\approx 72\text{--}74\%$) employ resource-intensive multi-candidate generation and full agentic loops [5]. PHEBE prioritizes real-time interactive latency, achieving 61.6% with a single-candidate, self-correcting pipeline that is suited to production web deployment.

2.4 Frontend Visualization Stacks

Comparisons of React visualization libraries identify Recharts, Victory, Nivo, and D3.js as dominant options [1]. D3.js is the most expressive for custom visual encodings, offering precise SVG control. Performance comparisons of API architectures [2] demonstrate that FastAPI with an asynchronous request model outperforms Flask by 38% with GraphQL, establishing its suitability for real-time LLM inference pipelines. PHEBE adopts D3.js for client-side rendering and FastAPI for the backend, reflecting this evidence.

3 System Architecture

PHEBE is built on a three-tier web architecture with clear separation between the frontend presentation layer, the backend application layer, and the data persistence layer. All tiers communicate over a RESTful HTTP/JSON contract.

Frontend. The client is developed with React 19, bundled with Vite 7, and styled with Tailwind CSS. All chart rendering is performed client-side using D3.js, which provides full SVG control. Large result sets are virtualized using TanStack Table to avoid rendering the full DOM.

Backend. The server is implemented in Python with FastAPI and an asynchronous request model. All AI capabilities — SQL generation, cleaning recommendations, chart selection, and security classification — are handled through the OpenAI API. Distinct route modules serve authentication, dataset management, data cleansing, and the NL2Vis conversation pipeline.

Data Layer. MySQL is the primary relational store. Each uploaded CSV is converted into a dedicated MySQL table; SQL queries are executed directly against these tables at query time. Dataset metadata, conversation history, and session state are also persisted in MySQL. Authentication is handled via Firebase Admin SDK.

4 Data Ingestion and Validation

Users upload CSV files through the web interface. Before storage, each file passes through a sequential eight-stage validation pipeline:

- (1) **Extension and size check.** Files must carry a .csv extension and fall within 10 bytes to 100 MB.
- (2) **Encoding detection.** A two-pass strategy first checks for Unicode BOMs (UTF-8, UTF-16, UTF-32 variants), then falls back to probabilistic detection via the chardet library on the first 10,000 bytes. Accepted encodings include UTF-8, ASCII, ISO-8859-1, Windows-1252, and UTF-16/32 families.

- (3) **Delimiter detection.** Python’s csv.Sniffer identifies the field separator from the first 8,192 bytes. Accepted delimiters are comma, tab, pipe, and semicolon.
- (4) **Row and column bounds.** The system enforces a maximum of 1,000,000 data rows and 100 columns, with a minimum of one of each.
- (5) **Column consistency.** Every data row is verified to contain the same number of fields as the header. Up to five inconsistent rows are surfaced in the error response.
- (6) **Header validation and sanitization.** Column names are checked for duplicates and whitespace-only entries, then sanitized: lowercased, spaces replaced with underscores, leading digits prefixed with col_. SQL reserved keywords (SELECT, DROP, etc.) trigger warnings rather than hard rejections.
- (7) **Headerless file handling.** When csv.Sniffer.has_header returns False, column names are auto-generated as Column 1, Column 2, etc.
- (8) **Type inference and persistence.** Pandas infers column data types; users may override inferred types before the DataFrame is written to MySQL. Dataset-level metadata (file-name, row count, column count, encoding, upload timestamp) is stored in a metadata table.

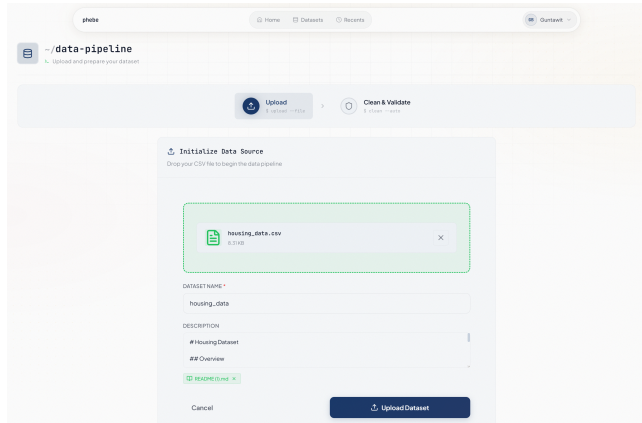


Figure 1: Data ingestion interface. Users drop a CSV file, set a dataset name and description, and the system auto-detects encoding, delimiter, and column types before committing to the database.

After upload, users may supply a free-text description or README file. This description is injected into every LLM prompt at query time, helping the model interpret domain-specific column names and expected value ranges.

5 AI-Assisted Data Cleaning

Rather than applying automatic bulk corrections, the cleansing subsystem presents data quality issues to the user one at a time, in a fixed priority order that reflects downstream dependencies. Format inconsistencies are detected first because the missing value detector produces less accurate results when string null-like values (e.g., “N/A”, “—”) remain un-normalized. The complete detection order

is: (1) format inconsistencies, (2) missing values, (3) outliers, (4) duplicate rows, (5) duplicate columns, (6) high-cardinality columns.

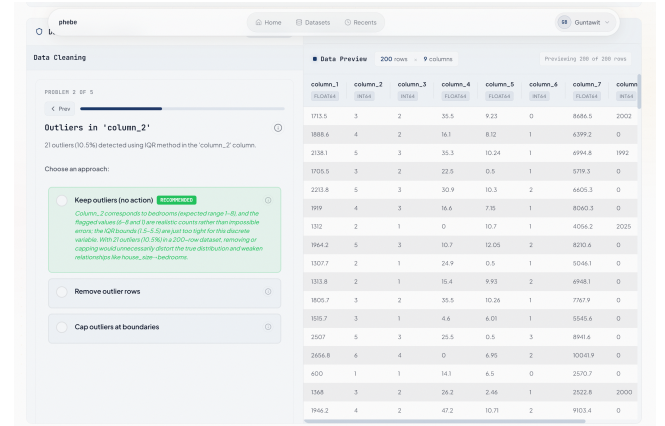


Figure 2: AI-assisted data cleaning interface. The system presents detected data quality issues one at a time (here: outliers in a numeric column), shows an AI-recommended action with rationale, and lets the user accept, choose an alternative, or skip.

5.1 Detection Methods

Missing Values. Columns with at least 1% null values are reported. Severity is CRITICAL (>50% missing), WARNING (20–50%), or INFO (1–20%). Remediation options include dropping the column, dropping affected rows, imputing with mean/median/mode, or filling with a user-specified constant.

Outlier Detection. Applied to numeric columns using the IQR method ($k = 1.5$):

$$\text{Lower} = Q_1 - 1.5 \cdot \text{IQR}, \quad \text{Upper} = Q_3 + 1.5 \cdot \text{IQR}$$

A column requires at least four non-null values. Severity is CRITICAL when outliers exceed 10% of values, WARNING otherwise. Options: remove outlier rows, cap at bounds, or retain.

Duplicate Detection. Rows are compared across all columns via pandas duplicated(); columns are compared element-wise via Series.equals(). Severity thresholds: CRITICAL for row duplicates >20%, WARNING for column duplicates with more than two redundant columns.

Format Inconsistencies. Checked across four sub-types on string/object columns: mixed numeric/text types, inconsistent date formats (eleven regex patterns including ISO 8601, European, US, and verbose formats), inconsistent boolean encodings (Yes/No, Y/N, True/False, 1/0, T/F, On/Off), and mixed text casing (UPPERCASE, lowercase, Title Case, mixed).

High Cardinality. Columns with $\geq 90\%$ unique values among non-null entries, at least 10 unique values, and at least 20 dataset rows are flagged. Identifier-pattern columns (suffixes _id, _key, _code, uuid, email, etc.) receive INFO severity; others receive WARNING.

5.2 AI Recommendations and Undo

For each detected problem, the LLM (at temperature 0.3) receives the problem description and available remediation options and returns its recommended action with a single-sentence rationale, which is shown alongside the option list. Recommendations are cached per problem_id to avoid duplicate API calls.

Each confirmed operation is applied to the in-memory pandas DataFrame and pushed onto a session-scoped stack. Undo pops the most recent entry and restores the DataFrame from a disk-backed snapshot. Backup files are cleaned up after a 30-minute idle timeout and by an hourly background job.

6 Text-to-Visualization Agent

The core user-facing capability is the text-to-visualization agent, composed of four cooperating LLM-powered components: the Orchestrator Agent, the Text-to-SQL Agent, the Analysis Agent, and the Chart Recommendation Agent (Figure 3).

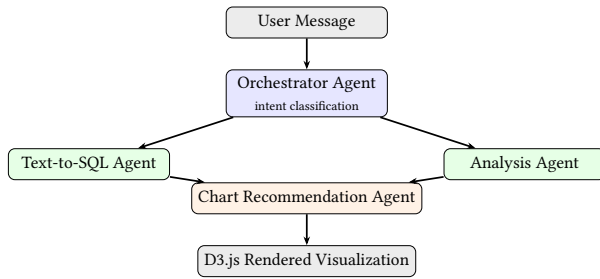
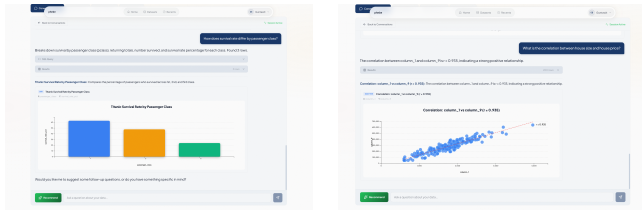


Figure 3: Multi-agent pipeline for text-to-visualization. The Orchestrator classifies intent and routes to either the Text-to-SQL Agent or the Analysis Agent; results flow to the Chart Recommendation Agent for visual encoding selection.



(a) Bar chart generated from a natural language question about Titanic survival rate by passenger class. (b) Scatter plot with Pearson $r = 0.935$ generated for a correlation query on the housing dataset.

Figure 4: Conversation interface showing AI-generated visualizations. The system translates natural language to SQL, executes it, and automatically selects the appropriate chart type.

6.1 Session Initialization and Schema Injection

When a user opens a dataset for conversation, a session is created and a schema context object is constructed from table metadata:

table name, column names and inferred types, up to three sample string values per column, row count, and user-provided description. This context is injected into every LLM system prompt, ensuring the model reasons over the actual dataset structure rather than hallucinating columns or value ranges.

The Orchestrator makes an initial LLM call to generate a conversational introduction and three to four recommended starter questions tailored to the specific dataset schema.

6.2 Conversation History Management

Each user and assistant message is persisted to the MySQL conversations table, including message content, role, executed SQL, result set, and visualization payload. Every Orchestrator call includes the most recent six messages from the conversation history, enabling multi-turn follow-up questions and anaphora resolution (e.g., “show that as a bar chart” referring to a prior result). Sessions are resumable from persistent storage without requiring re-upload.

6.3 Orchestrator Agent and Intent Classification

Every user message is first classified by the Orchestrator (temperature 0.3) into one of six intent categories:

- **SQL data query** — question answerable via SQL
- **Statistical analysis request** — requires correlation or feature-importance computation
- **Chart type change** — explicit request to re-render with a different chart
- **Recommendations request** — user asks for suggested exploration questions
- **Conversational** — general question about dataset structure or approach
- **Clarification needed** — ambiguous query requiring follow-up

The Orchestrator routes to the appropriate sub-agent or handles the response directly.

6.4 Text-to-SQL Agent and Validation

The Text-to-SQL Agent (temperature 0.3) generates a SELECT statement and an explanation from the schema context plus a comprehensive rule set: use only existing columns, apply LIMIT 1000, use MySQL-compatible syntax, and bucket continuous numeric columns in CASE WHEN ranges for GROUP BY to prevent visualizations with hundreds of bars.

Before execution, queries pass through a three-layer SQLVALIDATOR:

- (1) **Security validation:** whole-word regex blocking of DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE, GRANT, REVOKE, CREATE, EXEC, EXECUTE, MERGE, CALL, LOAD, IMPORT, and multi-statement semicolons.
- (2) **Syntax validation:** parsed via sqlparse; checks SELECT prefix, balanced parentheses, and matched quotes.
- (3) **Schema validation:** extracted identifiers are checked against known column names, with fuzzy matching (SequenceMatcher threshold 0.6) for typo suggestions.

6.5 Execution-Based Self-Correction

If a validated query fails during MySQL execution, the system enters a self-correction loop: the exact database error message is appended to a new LLM call alongside the original query and schema. This repeats for up to three attempts. The loop also handles empty result sets (likely over-restrictive WHERE) and all-NULL results (likely incorrect column references) as distinct failure modes.

6.6 Analysis Agent

The Analysis Agent handles statistical queries that SQL cannot express. It supports three analysis types:

Factor Impact Analysis. Uses a Random Forest (scikit-learn, 100 estimators, max depth 6) to rank column influence on a target. RandomForestClassifier is used when the target has ≤ 20 unique values; RandomForestRegressor for continuous targets. High-cardinality identifier columns (unique ratio $> 50\%$) are excluded automatically.

Correlation Analysis. Selects the appropriate measure by column data types: Pearson’s r for two numeric columns (scatter plot, up to 500 sampled points); point-biserial correlation for one categorical and one numeric (grouped bar chart); Cramér’s V for two categorical columns, computed as:

$$V = \sqrt{\frac{\chi^2}{n \cdot (\min(r, c) - 1)}}$$

where r and c are contingency table dimensions. Correlation strength labels: strong ($|r| \geq 0.7$), moderate (≥ 0.4), weak (≥ 0.2), very weak otherwise.

6.7 Chart Recommendation Agent

After every successful SQL execution, the Chart Recommendation Agent (temperature 0.2 for maximum consistency) selects among bar, line, pie, scatter, area, and histogram charts based on the result set’s column names, types, and a data sample. Single-value scalar results are rendered as a table. The response is formatted as a D3.js-compatible JSON object with axis specifications, a suggested title, and a justification. When the user explicitly requests a chart type, it is passed as `preferred_chart_type`, overriding the model’s selection.

6.8 Security Guardrails

Three independent validators protect the pipeline against prompt injection and destructive SQL. A **Prompt Injection Guard** screens every incoming message with an LLM classifier (temperature 0.0) that returns only “yes” or “no”, rejecting attempts to override system instructions while passing all legitimate queries. An **Indirect Injection Guard** applies synchronous regex patterns to schema content before it reaches LLM prompts, then submits it to a secondary classifier to catch obfuscated variants embedded in CSV data. A **SQL Safety Guard** performs a final synchronous regex check for destructive keywords immediately before MySQL execution. All validators fail open to preserve availability when AI classifier APIs are unreachable; only the SQL Safety Guard is synchronous and always enforced.

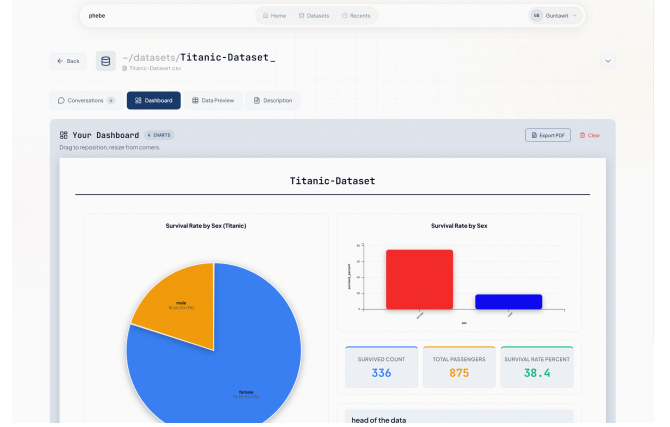


Figure 5: Dashboard interface showing pinned visualizations from multiple queries. Users can drag and reposition charts, customize colors, and export the entire dashboard as a PDF.

7 Dashboard and PDF Export

Each successfully executed query automatically generates a chart in the conversation panel. Users can pin any chart to a persistent dashboard by clicking the Pin button. Within the dashboard, charts are arranged via drag-and-drop; individual chart titles and the dashboard title are editable in-place; and color schemes are customizable per chart. The completed dashboard can be exported as a PDF document capturing all charts in their current arrangement and styling.

8 Evaluation

8.1 BIRD-Mini Benchmark

SQL generation accuracy was evaluated on the BIRD Mini Dev set [5], a standard benchmark of 500 natural language question–SQL pairs spanning multiple real-world database schemas at three difficulty levels (simple, moderate, challenging), evaluated by execution accuracy (EX): a predicted query is correct if its result set matches the gold standard when both are executed against the same database.

The evaluation pipeline implements four stages:

- (1) **Enriched schema construction.** Per-database: CREATE TABLE DDL, row counts, up to three sample values per column (or distinct-value counts for high-cardinality numerics), and a full foreign key map. Cached per database.
- (2) **Few-shot SQL generation.** Five manually curated examples covering the most common BIRD failure modes: unnecessary JOINS, incorrect LOWER() wrapping, missing CAST for ratios, over-restrictive filters, and incorrect date extraction.
- (3) **Execution-based self-correction.** Up to three retry rounds on execution errors, empty results, and all-NULL results.
- (4) **LLM verification pass (optional).** A second LLM call reviews generated SQL against six error-type checklists and may propose a revised query, which is executed before replacing the original.

Results. Table 1 summarizes performance against baselines. PHEBE achieves 61.6% overall EX, an 8.0 percentage point improvement over the GPT baseline. Performance broken down by difficulty: simple 72.97% (108/148), moderate 58.80% (147/250), challenging 51.96% (53/102). Among the 100 questions where the pipeline and baseline disagreed, the pipeline was correct in 70 cases. The self-correction loop fixed 30 out of 500 questions (6% of the full test set) and reduced syntax errors from 11 (baseline) to 0, achieving a 0% SQL syntax error rate.

Table 1: BIRD-Mini Benchmark Execution Accuracy. SOTA figures sourced from the official BIRD leaderboard [5].

System	EX (%)	vs. Baseline
GPT Baseline (zero-shot)	53.6%	—
PHEBE (optimized pipeline)	61.6%	+8.0%
SuperSQL (SOTA)	≈72–74%	reference
CHASE-SQL / Apple (SOTA)	≈73%	reference
Distillery / Google (SOTA)	≈75–77%	reference

8.2 User Survey

A controlled user survey ($n = 27$ participants) was conducted to evaluate the usability of the visualization interface. The participant pool comprised undergraduate students (21), graduate students (4), a high school student (1), and a UX/UI practitioner (1) — reflecting a range of technical backgrounds rather than specialist data practitioners. Self-reported SQL and data analysis proficiency averaged 3.2 out of 5, and the majority used BI tools monthly or less frequently (rarely: 8, monthly: 7, weekly: 10, daily: 2), confirming the target non-technical demographic. Each participant was given a structured dataset (Titanic passenger records) and asked to complete four tasks using only the natural language interface, without writing any SQL or code.

The survey instrument comprised three sections: background questions (Q1–3) capturing role, data proficiency, and BI tool frequency; task-level evaluation (Q4–7) rating dataset upload, AI-guided data cleaning, chart generation, and dashboard/export experience on a five-point Likert scale; and a deeper evaluation section (Q8–14) probing cleaning agent effectiveness, chart quality and aesthetics, SQL-free exploration ability, chart suitability, error message clarity, overall UI experience, and overall system experience, with an open-ended comment field.

Results. Table 2 summarizes mean Likert scores across all rated dimensions. Upload succeeded for 25 of 27 participants (93%); 2 encountered errors. The NL-to-visualization task was completed on the first attempt by 14 participants (52%), required rephrasing by 12 (44%), and failed for 1 (4%). The cleaning agent caught all data issues for 19 participants (70%), missed some for 6 (22%), and made unexpected changes for 1 (4%). Error messages were rated very helpful by 16 participants (59%) and somewhat helpful by 9 (33%).

On the key accessibility metric, 85% of participants (23 of 27) rated their ability to explore the dataset without SQL or programming at 4 or 5 out of 5, with a mean score of 4.33 — substantially

validating the platform’s goal of democratizing data exploration for non-technical users. The highest-rated dimension was overall UI experience (4.52/5). Open-ended comments reflected three recurring themes: appreciation for the ease of use, uncertainty about what cleaning recommendations meant (e.g., “fill missing values with median”), and occasional dissatisfaction with the automatically selected chart type.

Table 2: Mean Likert Scores by Evaluation Dimension ($n = 27$, scale 1–5)

Evaluation Dimension	Mean
Task 2: AI-guided data cleaning (Q5)	4.26
Task 4: Dashboard customization and export (Q7)	4.26
Chart quality, clarity, and aesthetics (Q9)	4.33
SQL-free dataset exploration (Q10)	4.33
Chart suitability for query (Q11)	4.22
UX/UI overall experience (Q13)	4.52
Overall system experience (Q14)	4.30

9 Conclusion

We presented PHEBE, a multi-agent web platform enabling non-technical users to upload, clean, query, and visualize tabular data using natural language. The system’s four-agent architecture achieves 61.6% execution accuracy on the BIRD-Mini benchmark (+8% over baseline) with a 0% SQL syntax error rate, and a user survey ($n = 27$) confirms that 85% of participants rated SQL-free dataset exploration at 4 or 5 out of 5, with an overall mean experience rating of 4.30/5. Key innovations include a dependency-ordered six-category data cleaning pipeline with stack-based undo, an execution-based self-correction loop that handles three distinct failure modes, and a multi-layer security guardrail architecture tailored for user-uploadable data. Future work will focus on multi-candidate SQL generation to close the gap with SOTA leaderboard systems while maintaining interactive latency, improved natural language explanations for cleaning recommendations, and a supervisor agent to further automate internal validation before results are shown to users.

References

- [1] Capital One Tech. 2024. A Comparison of Data Visualization Libraries for React. Capital One Tech Blog. (2024). <https://www.capitalone.com/tech/software-engineering/comparison-data-visualization-libraries-for-react/> Software Engineering & Web Development.
- [2] Dennis Demir and Edward Nilsson. 2023. Performance comparison of REST vs GraphQL in different web environments - Node.js and Python. Computer Science.
- [3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Ychen Qian, Bolin Ding, and Jingren Zhou. 2023. DAIL-SQL: Efficient Prompt Engineering for Large Language Model Text-to-SQL. *Proceedings of the VLDB Endowment* 17, 2 (2023), 148–161. doi:10.14778/3626292.3626301
- [4] Yuhang Ge, Yachuan Liu, Yuren Mao, and Yunjun Gao. 2025. Text-to-Pipeline: Bridging Natural Language and Data Preparation Pipelines. *arXiv preprint arXiv:2505.15874* (May 2025). doi:10.48550/arXiv.2505.15874 Applied Science & Technology Source Ultimate.
- [5] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can LLM Already Serve as a Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates.

- [6] Dominik Moritz, Chenglong Wang, Greg L. Nelson, Halden Lin, Adam M. Smith, Bill Howe, and Jeffrey Heer. 2019. Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco. In *IEEE Transactions on Visualization and Computer Graphics*, Vol. 25. IEEE, 438–448. doi:10.1109/TVCG.2018.2865240
- [7] Arpit Narechania, Arjun Srinivasan, and John Stasko. 2021. NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries. In *IEEE Transactions on Visualization and Computer Graphics*, Vol. 27. IEEE, 369–379. doi:10.1109/TVCG.2020.3030378
- [8] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates.
- [9] Canwen Xu, Julian McAuley, and Penghan Wang. 2023. Mirror: A Natural Language Interface for Data Querying, Summarization, and Visualization. (March 2023). arXiv:2303.08697 doi:10.1145/3543873.3587309 arXiv preprint arXiv:2303.08697; Applied Science & Technology Source Ultimate.
- [10] Bowen Yu and Claudio T. Silva. 2020. FlowSense: A Natural Language Interface for Visual Data Exploration within a Dataflow System. In *IEEE Transactions on Visualization and Computer Graphics*, Vol. 26. IEEE, 1–11. doi:10.1109/TVCG.2019.2934668
- [11] Weixu Zhang, Yifan Shen, Yuanfeng Huang, Yueting Wang, Zhenhui Li, Yun Raymond Chen, Dong Jiang, Wei Wang, and Jiawei Han. 2024. Natural Language Interfaces for Tabular Data Querying and Visualization: A Survey. *arXiv preprint arXiv:2310.17894* (May 2024). arXiv:2310.17894 [cs.CL] Natural Language Processing & Data Visualization.
- [12] Ruikang Zhou and Fan Zhang. 2025. Refining Zero-Shot Text-to-SQL Benchmarks via Prompt Strategies with Large Language Models. *Applied Sciences* 15, 10 (May 2025). Applied Science & Technology Source Ultimate.